

Databases – Part I

Objectives:

In this lab, student will be able to

1. Understand the MTV architecture
2. Create an App in Django and establish a connection with SQLite database
3. Set different privileges to different types of users.
4. Set the Django administrator account.

Django supports following databases

1. MySql
2. PostgreSQL
3. Oracle
4. Sqlite

With the help of 3rd party backend Django supports following databases

1. SAP SQL Anywhere
2. IBM DB2
3. Microsoft SQL Server
4. Firebird
5. ODBC
6. ADSDB

Django abstracts the details of underlying database. One only need to specify the (models.py) python functions which will be converted into underlying database statements. Django supports CRUD operations. There are two way one can control the data on the website. First way is to use the admin interface second way is to use the forms.

Solved Exercise

Model is the name given to data abstraction part. To create the model you must first create an app. To create an app right click on the project → Add → DjangoApp
Let us name the app as “blog”

Step1: In settings.py add the app name (blog) under Installed_Apps as follows:

```
INSTALLED_APPS = [
    # Add your apps here to enable them
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'blog'
]
```

Under Templates provide the path of the template directory as follows

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'blog/templates/blog')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

If you are using sqlite leave the default setting for database which will look as follows

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),  
    }  
}
```

If you are using MySQL in that case modify the database entry as follows

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'OPTIONS': {'read_default_file': '/path/to/my.cnf'},  
    }  
}
```

My.cnf file is as follows:

```
# my.cnf  
[client]  
database = NAME  
  
user = USER  
  
password = PASSWORD  
default-character-set = utf8
```

Step2: Modify the Projects urls.py as given below from

```
django.conf.urls import include, url

# Uncomment the next two lines to enable the admin:

from django.contrib import admin admin.autodiscover()

urlpatterns = [
    # Examples:
    # url(r'^$', MyBlog.views.archive, name='archive'),
    #url(r'^MyBlog/', include('MyBlog.MyBlog.urls')),
    # Uncomment the admin/doc line below to enable admin documentation:
    # url(r'^admin/doc/', include('django.contrib.admindocs.urls')),

    # Uncomment the next line to enable the admin:
    # (r'^$', 'django.views.generic.simple.redirect_to',
    # {'url': '/blog/'}), url(r'^blog/',

include('blog.urls')), url(r'^admin/',

include(admin.site.urls)),

]
```

Step3: Under blog app create a file named urls.py and type the following

```
from django.conf.urls import include,url from blog.views import

archive,create_blogpost

urlpatterns = [ url(r'^$', archive, name='archive'),

url(r'^create/', create_blogpost, name='create_blogpost'),

]
```

Step4: Under models.py type the following from

```
django.db import models
```

```
# Create your models here. from
```

```
django import forms class
```

```
BlogPost(models.Model):
```

```
title
```

```
models.CharField(max_length=
```

```
150) body =
```

```
models.TextField()
```

```
timestamp =
```

```
models.DateTimeField()
```

```
class Meta:
```

```
ordering = ('-timestamp',)
```

```
class BlogPostForm(forms.ModelForm):
```

```
class Meta:
```

```
model = BlogPost
```

```
exclude = ('timestamp',)
```

It contains the details of table and Model form uses the model already created to create the form. This approach avoids duplication of code and goes with python philosophy Do not Repeat Yourself.

Step5: Registering your app in the admin: To register your app type the following into

```
admin.py from django.contrib import admin import site from blog.models import
```

```
BlogPost
```

```
# Register your models here. from blog
import models class
BlogPostAdmin(admin.ModelAdmin):
    list_display = ('title', 'timestamp')
admin.site.register(models.BlogPost,BlogPostAdmin)
```

Step6: Type the following into views.py from

```
django.shortcuts import render

# Create your views here. from datetime import
datetime from django.http import
HttpResponseRedirect from django.shortcuts
import render from blog.models import BlogPost,
BlogPostForm
def archive(request):
    posts = BlogPost.objects.all()[:10]
    return render(request, 'archive.html',
                  {'posts': posts, 'form': BlogPostForm()})
def create_blogpost(request):
    if request.method == 'POST':
        form = BlogPostForm(request.POST)
        if form.is_valid():
```

```

        post = form.save(commit=False)

post.timestamp=datetime.now()

        post.save()

    return HttpResponseRedirect('/blog/')

```

It is displaying the 10 most recent blogs posted by users/admin.

Step7: Type the following lines into archive.html

```
<!DOCTYPE html>
```

```
<html lang="en" xmlns="http://www.w3.org/1999/xhtml">
```

```
<head>
```

```
    <meta charset="utf-8" />
```

```
    <title></title>
```

```
</head>
```

```
<body>
```

```
<form action="/blog/create/" method=post>{% csrf_token %}
```

```
    <table>{{ form }}</table><br>
```

```
    <input type=submit>
```

```
</form>
```

```
<hr>
```

```
{% for post in posts %}
```

```
    <h2>{{ post.title }}</h2>
```

```
    <p>{{ post.timestamp }}</p>
```

```
    <p>{{ post.body }}</p>
```

```
<hr>
```

```
{% endfor %}
```

```
</body> </html>
```

This is the template which displays the blog posts that are separated by horizontal rule.

Once you have typed all of the above

- i. Go to Projects→Django Check

If it succeeds then

- ii. Goto Projects→Django Make Migrations

If it succeeds then

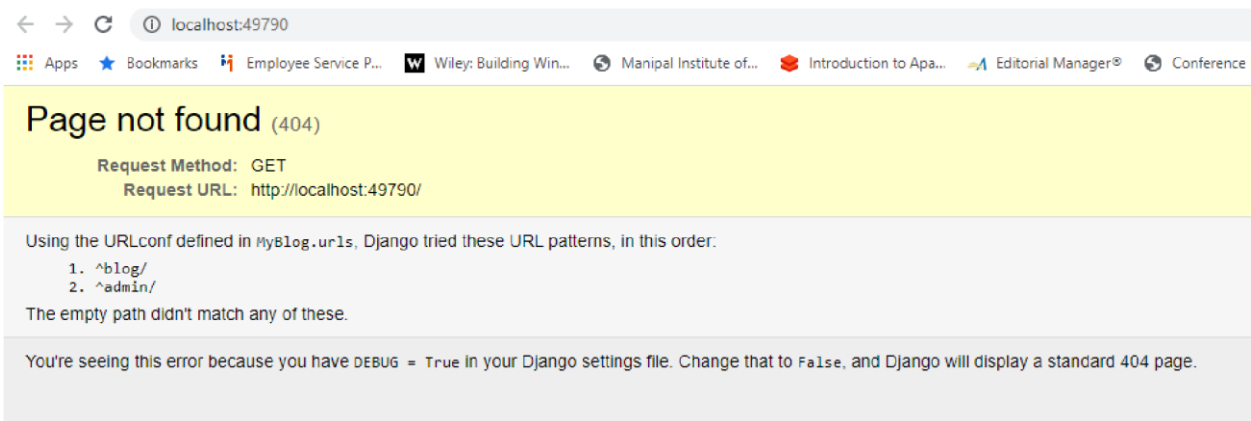
- iii. Goto Projects→Django Migrate

If it Succeeds then

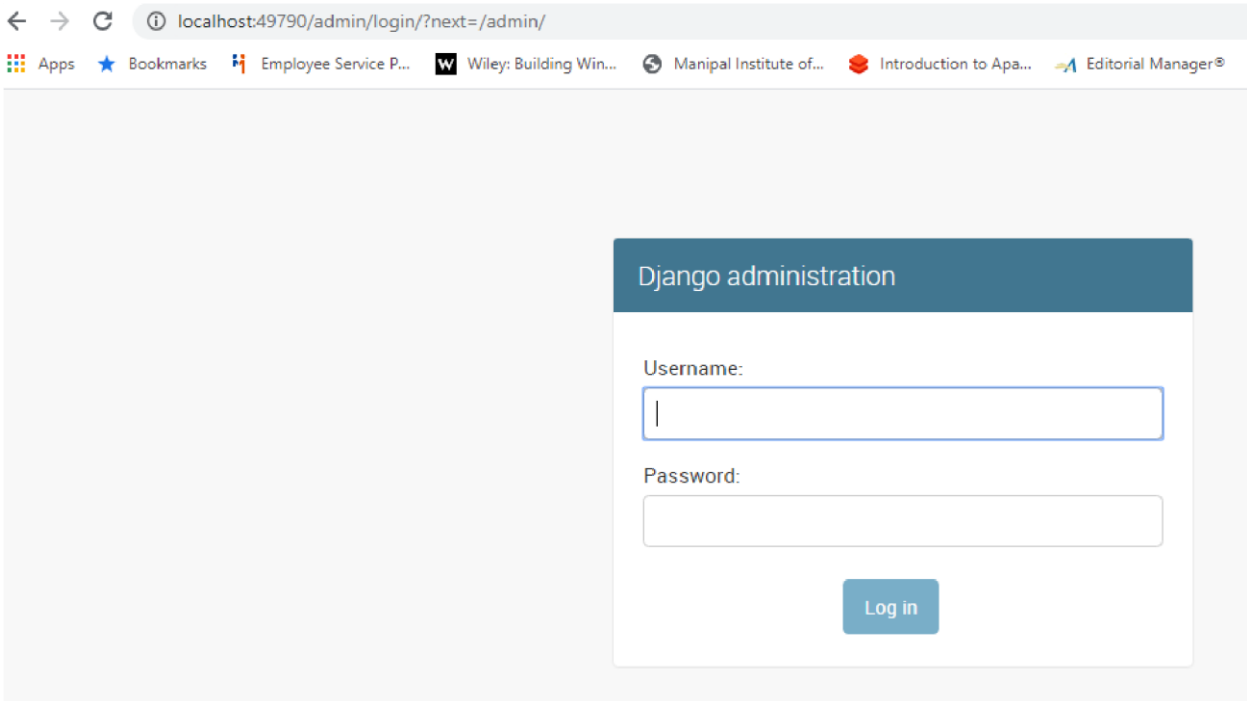
- iv. Goto Projects→Django Create Superuser

You have to repeat the above four steps whenever you modify the model or use different database.

Once you have created the superuser you can open the website. It looks as follows



append /admin to the host name you will get the following screen



← → ↻ ⓘ localhost:49790/admin/login/?next=/admin/

Apps ★ Bookmarks ⓘ Employee Service P... W Wiley: Building Win... ⓘ Manipal Institute of... ⓘ Introduction to Apa... ⓘ Editorial Manager®

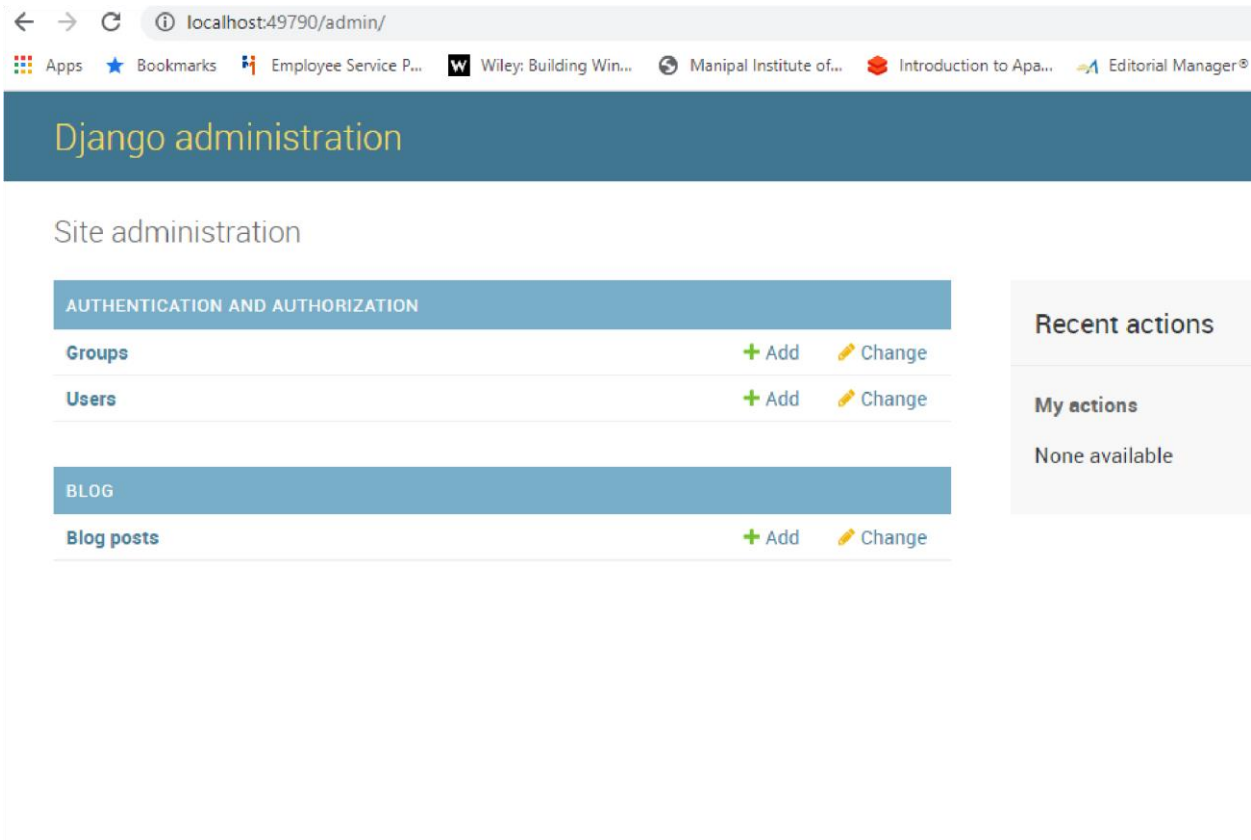
Django administration

Username:

Password:

Log in

Type the superuser name and password you will be taken to following admin page. In the admin page you can see entry for Blog Posts as you have registered it.



The screenshot displays the Django administration interface. The browser's address bar shows 'localhost:49790/admin/'. The page title is 'Django administration'. Under the 'Site administration' heading, there are two main sections: 'AUTHENTICATION AND AUTHORIZATION' and 'BLOG'. The 'AUTHENTICATION AND AUTHORIZATION' section contains 'Groups' and 'Users', each with '+ Add' and 'Change' links. The 'BLOG' section contains 'Blog posts' with '+ Add' and 'Change' links. On the right sidebar, 'Recent actions' is empty, and 'My actions' shows 'None available'.

Add a blog post You will be taken to following screen you can observe only title and timestamp are visible as per our code

localhost:49790/admin/blog/blogpost/

Apps Bookmarks Employee Service P... Wiley: Building Win... Manipal Institute of... Introduction to Apa... Editorial Manag

Django administration

Home › Blog › Blog posts

✔ The blog post "BlogPost object" was added successfully.

Select blog post to change

Action: 0 of 1 selected

☐	TITLE	TIMESTAMP
☐	Internet Technology Lab	May 10, 2020, 10:50 a.m.

1 blog post

Now you append the blog to the address you will get the following output.

    localhost:49790/blog/

 Apps  Bookmarks  Employee Service P...  Wiley: Building Win... 

Title:

Body:

Internet Technology Lab

May 10, 2020, 10:50 a.m.

Welcome to the Lab

As per our instructions users cannot edit the timestamp and current date and time will be taken for user entry. The blog entered by admin is also displayed. Once you enter the blog post details it will display it as under.

 localhost:49790/blog/

 Apps  Bookmarks  Employee Service P...  Wiley: Building Win...  Manipal Institute

Title:

Body:

Internet Technology Lab

May 10, 2020, 4:29 p.m.

Do the lab excercises

Internet Technology Lab

May 10, 2020, 10:50 a.m.

Welcome to the Lab

LAB EXERCISES

1. Design a web site using Django, which is a website directory – A site containing links to other websites. A web page has different categories.
 - A category table has a name, number of visits, and number of likes.
 - A page table refers to a category, has a title, URL, and many views.

Design a form that populates the above database and displays it.

2. Consider the following tables:

WORKS(person-name,Company-name,Salary)

LIVES(Person_name, Street, City)

Assume Table data suitably. Design a Django webpage and include an option to insert data into WORKS table by accepting data from the user using TextBoxes. Also, include an option to retrieve the names of people who work for a particular company along with the cities they live in (particular company name must be accepted from the user).

ADDITIONAL EXERCISES

1. Assume a table “Institutes” with institute_id, name, and no_of_courses are the fields. Create a web page that retrieves all the data from “Institutes” table displays only Institute names in the list box.

Databases-Part II

Lab Exercise

1. There are three tables in the database an author table has a first name, a last name and an email address. A publisher table has a name, a street address, a city, a state/ province, a country, and a Web site. A book table has a title and a publication date. It also has one or more authors (a many-to-many relationship with authors) and a single publisher (a one-to-many relationship - aka foreign key - to publishers). Design a form which populates and retrieves the information from the above database using Django.
2. Create a Django Page for entry of a Product information (title, price and description) and save it into the db. Create the index page where you would view the product entries in an unordered list.
3. Create a web page with DropDownList, Textboxes and Buttons. Assume the table 'Human' with First name, Last name, Phone, Address and City as fields. When the page is loaded, only first names will be displayed in the drop-down list. On selecting the name, other details will be displayed in the respective TextBoxes. On clicking the update button, the table will be updated with new entries made in the text box. On clicking the delete button, the selected record will be deleted from the table, and the DropDownList is refreshed.

ADDITIONAL QUESTIONS

1. Create a web page that receives the following information from a set of students: Student Id, Student Name, Course Name and Date of Birth. The application should also display the information of all the students once the data is Entered.